

2. Develop a C program for processing sparse matrices using an appropriate data structure. The program shall accept suitable sparse matrix data, use an appropriate representation, and perform operations such as addition and transpose. Demonstrate the correctness of the solution using suitable test cases.

```
#include <stdio.h>

#define MAX 10

void readMatrix(int a[MAX][MAX], int *r, int *c)
{
    int i, j;

    printf("Enter number of rows and columns: ");
    scanf("%d%d", r, c);

    printf("Enter matrix elements:\n");
    for(i = 0; i < *r; i++)
        for(j = 0; j < *c; j++)
            scanf("%d", &a[i][j]);
}

void display(int a[MAX][MAX], int r, int c)
{
    int i, j;

    for(i = 0; i < r; i++)
    {
        for(j = 0; j < c; j++)
            printf("%d\t", a[i][j]);

        printf("\n");
    }
}

void add(int a[MAX][MAX], int b[MAX][MAX],
        int c[MAX][MAX], int r, int col)
{
    int i, j;

    for(i = 0; i < r; i++)
        for(j = 0; j < col; j++)
            c[i][j] = a[i][j] + b[i][j];
}

void transpose(int a[MAX][MAX], int t[MAX][MAX],
              int r, int c)
{
    int i, j;
```

```

    for(i = 0; i < r; i++)
        for(j = 0; j < c; j++)
            t[j][i] = a[i][j];
}

int main()
{
    int a[MAX][MAX], b[MAX][MAX];
    int sum[MAX][MAX], t[MAX][MAX];
    int r1, c1, r2, c2;

    printf("Enter Matrix A\n");
    readMatrix(a, &r1, &c1);

    printf("\nEnter Matrix B\n");
    readMatrix(b, &r2, &c2);

    printf("\nMatrix A:\n");
    display(a, r1, c1);

    printf("\nMatrix B:\n");
    display(b, r2, c2);

    if(r1 == r2 && c1 == c2)
    {
        /* Addition */
        add(a, b, sum, r1, c1);

        printf("\nAddition of A and B:\n");
        display(sum, r1, c1);

        /* Transpose of SUM */
        transpose(sum, t, r1, c1);

        printf("\nTranspose of Sum:\n");
        display(t, c1, r1);
    }
    else
    {
        printf("\nAddition not possible\n");
    }

    return 0;
}

```